Bridge Technical Operations Runbook

Pilot operations runbook | Version 1.0 | 2 November 2026

Purpose and limits

This is a stack-neutral runbook template for operating the Bridge pilot. No deployed service, cloud provider, monitoring platform, database, or pager process was supplied. Before production use, the technical owner must fill in actual service URLs, dashboards, alert destinations, contacts, deployment/rollback commands, restore steps, recovery targets, and smoke checks. This document does not claim those systems currently exist.

Service map to complete

Component	Actual name / owner	Health signal / link
Mobile client	To be assigned	Release/version and crash signal
API service	To be assigned	Availability, error rate, latency
Identity provider	To be assigned	Login and token validation health
Primary datastore	To be assigned	Connections, storage, replication, backup status
Analytics/events	To be assigned	Ingestion delay and data quality
Messaging delivery (if realtime)	Not defined	Required only if realtime is selected

Operational readiness before pilot

- Name a technical incident lead and backup.
- Set coverage hours and escalation route.
- Establish service dashboards for availability, errors, latency, database health, and authentication failures.
- Confirm alert routing and test a non-production alert.
- Document deploy, rollback, backup, and restore procedures for the selected stack.
- Set recovery point objective (RPO), recovery time objective (RTO), and data retention with the business owner.
- Ensure sensitive tokens, credentials, profile details, and message bodies are excluded from routine logs.
- Verify the support and trust owners know how to report a service incident.

Severity model

Severity	Example	Response
SEV-1	Broad outage, suspected unauthorized data access, corrupted task/application states.	Incident lead immediately coordinates containment, user-impact assessment, and updates.
SEV-2	Major core flow unavailable for a meaningful user group, or repeated message/application failures.	Assign lead, investigate, consider rollback/workaround, and provide status updates.
SEV-3	Limited defect with workaround and no confirmed data-integrity/security impact.	Create ticket, assess scope, schedule fix.
SEV-4	Cosmetic or low-impact issue.	Add to backlog and review in normal planning.

Actual paging coverage and response targets must be agreed; severity labels do not imply round-the-clock staffing.

Incident workflow

1. **Detect:** confirm alert or report using service health signals and request IDs.
2. **Declare:** choose severity, name incident lead, and open a restricted incident record.
3. **Scope:** identify affected services, users, time range, data objects, and current risk. Avoid destructive remediation before preserving necessary evidence.
4. **Mitigate:** use the least risky available action: rollback known bad release, disable affected feature, scale/restart only when evidence supports it, or pause writes if data integrity is at risk.
5. **Communicate:** support owner shares an approved status update and next update time; do not speculate about cause.

6. **Recover:** verify core journeys, record integrity, and monitoring before declaring service restored.
7. **Review:** capture timeline, cause, impact, actions, and follow-up owners; avoid blame-focused reporting.

Common playbooks

Login failure spike

- Check identity provider status and recent auth configuration/deploy changes.
- Compare provider errors with API validation errors; do not log or request user passwords/tokens.
- If a recent configuration change is implicated, use the approved rollback path.
- Ask support to share user-facing guidance only after a workaround is verified.

Task/application write failures

- Check API error rate, datastore health, and recent schema/deploy changes.
- Determine whether failures are rejected writes or ambiguous timeouts before asking users to retry.
- Verify idempotency and state integrity for applications and accept/reject decisions.
- Pause affected writes if duplicate or inconsistent decisions are possible; notify marketplace operations.

Messaging failures

- Verify conversation access checks and message persistence separately from delivery/notification.
- Confirm whether the product promises realtime or only refresh-based updates.
- Do not inspect message content unless necessary and authorized for incident scope.
- Confirm recovery by sending and reading test messages with two authorized test accounts.

Suspected data exposure

- Escalate to incident lead and trust owner immediately.
- Restrict the affected route or access path if safe and available.
- Preserve relevant access logs with limited access; avoid copying sensitive content into general channels.
- Determine affected data and time range; obtain qualified guidance for any external notification decisions.

Datastore degradation or restore

- Check connection saturation, storage, replication, and backup status.
- Do not run ad hoc destructive database changes during incident response.
- Follow the selected provider's documented failover/restore procedure and approved RPO/RTO.
- After recovery, verify counts and sample task/application/message records against known expected states.

Deploy and rollback checklist

Before deploy: reviewed change, migration plan, backup/rollback path, health dashboards, owner on-call, and smoke scenarios. After deploy: verify login, profile read/write, task browse/create, application submit/decision, and conversation access/send. Roll back when the release causes material core-flow failure or integrity/security risk and rollback is safe. Record the decision and verify service recovery.

Backup and recovery

Document backup frequency, encryption, retention, restore permissions, cross-region strategy if any, and last successful restore test. A backup is not considered operationally useful until restore has been tested. Define RPO/RTO with the business owner before launch; no values are assumed here.

Monitoring and logs

Recommended signals: API request rate, error rate, latency, auth success/failure, datastore connections/latency, task/application state-transition errors, message persistence errors, background job failures if introduced, and service health. Logs should use request IDs and object identifiers only where needed; redact credentials and personal/message content.

Post-incident review

Within the team's agreed review window, write: summary; impact; timeline; detection; contributing factors; what helped/hindered; recovery; follow-up actions with owners/dates; and any needed product/support/trust update. Track actions to completion.